DATA VORTEX | ROUND 2 TECHNICAL REPORT

Participant details:

Team Name: Joy

Team Size: 1

Team Head: Thathi Reddy Jivan Reddy

1. Problem Definition

Social media platforms produce a large amount of textual information that contains opinions, reactions, feedback and discussions. Understanding the sentiment expressed in these posts is important for the rebuilding of the semantic and comprehension layer of a social analytics system.

The goal of this project is to develop an NLP-based sentiment classification system that can automatically classify social media text into three sentiment categories: Positive, Negative and Neutral.

The solution is composed of a labelled textual dataset. The solution is built on an end-to-end NLP pipeline including data auditing, text preprocessing, TF-IDF feature extraction, machine learning model training, evaluation, confusion matrix analysis and error analysis.

The same TF-IDF representation was used to evaluate three machine learning approaches, namely Logistic Regression, Linear SVM and Multinomial Naive Bayes. The final model was chosen based on measured test performance on a held-out test set.

2. Preprocessing Pipeline

Social media text often contains URLs, user mentions, hashtags, HTML entities, inconsistent spacing, capitalization, and informal language. The preprocessing pipeline was designed to normalize these patterns while preserving information that may contribute to sentiment classification.

1. HTML Entity Decoding

HTML entities such as *&* were converted into their corresponding characters.

2. URL Normalisation

URLs were replaced with a standard *<URL>* token to reduce unnecessary vocabulary variation.

3. User Mention Normalisation

User mentions such as *@user* were replaced with *<USER>* .

4. Hashtag normalisation

The # symbol was removed while preserving the actual hashtag word.

Example:

#Cognos → cognos

#BI → bi

5. Whitespace Normalisation

Multiple spaces and unnecessary whitespace were converted into single spaces.

6. Lowercase Conversion

Text was converted to lowercase to reduce duplicate vocabulary caused by capitalization differences.

7. Negation and Punctuation Preservation

Negation-related words and punctuation were not aggressively removed because they can provide useful sentiment information. For example, expressions containing words such as not were preserved.

8.Duplicate Handling

Duplicate texts were not directly deleted from the dataset because the duplicated records had consistent sentiment labels. Instead, a GroupShuffleSplit strategy was used with the cleaned text as the grouping variable. This ensured that identical cleaned texts could

not appear in both the training and test sets, reducing the risk of data leakage.

Pipeline Flow

Raw Social Media Text

↓

HTML Entity Decoding

↓

URL Normalization

↓

User Mention Normalization

↓

Hashtag Normalization

↓

Whitespace Normalization

↓

Lowercase Conversion

↓

Negation & Punctuation Preserved

↓

Leakage-Safe Train/Test Split

↓

TF-IDF Feature Extraction

TF-IDF Feature Extraction

After preprocessing, the cleaned text was converted into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF). TF-IDF assigns higher importance to terms that are informative within a document but less common across the complete corpus.

The vectorizer was configured with unigrams and bigrams *(ngram_range=(1,2))* so that both individual words and two-word expressions could contribute to sentiment classification.

A minimum document frequency of 2 was used to remove extremely rare terms, while a maximum document frequency of 95% was applied to reduce the influence of terms appearing in almost all documents. Sublinear TF scaling was also enabled.

Importantly, the TF-IDF vectorizer was fitted only on the training data and subsequently used to transform the test data. This prevents information from the test set from influencing feature construction.

Feature Extraction Result

Parameter	**Value**
Feature method	TF-IDF
N-grams	Unigrams + Bigrams
Min document frequency	2
Max document frequency	0.95
Sublinear TF	Enable

Training samples	7,183
Test samples	1,817
Vocabulary/Features	25,650

3. Training Methodology

Three supervised machine learning algorithms were trained using the same TF-IDF feature representation: Logistic Regression, Linear Support Vector Machine (SVM), and Multinomial Naive Bayes.

A group-aware 80/20 train-test split was used. The cleaned text was used as the grouping variable so that identical posts could not be distributed across both training and testing sets.

Each model was trained using the same training feature matrix and evaluated on the same unseen test set, enabling a consistent comparison of model performance.

Models Evaluated :

Logistic Regression

• Used as a linear baseline for multiclass text classification.

• *max_iter=1000*.

Linear SVM

• Evaluated as another linear classifier suitable for sparse TF-IDF features.

• *C=1.0.*

Multinomial Naive Bayes

• Evaluated as a probabilistic text-classification approach.

• Default *MultinomialNB* configuration was used.

Model Comparison

Three machine learning models were evaluated using the same TF-IDF representation and the same held-out test set. Accuracy, weighted precision, weighted recall, and weighted F1-score were used for comparison.

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1
Logistic Regression	58.83%	58.97%	58.83%	58.88%
Linear SVM	57.57%	57.68%	57.57%	57.62%
Multinomial Naive Bayes	**59.49%**	**59.43%**	**59.49%**	**59.25%**

4. Model Selection & Justification

Based on the measured results, Multinomial Naive Bayes was selected as the final model among the three evaluated configurations. It achieved the highest test accuracy of 59.49% and the highest weighted F1-score of 59.25%.

The model was evaluated using the same TF-IDF features and held-out test data as the other models, providing a consistent basis for

comparison. Multinomial Naive Bayes was therefore retained as the final sentiment classification model for the implemented pipeline.

Why this model was retained

Highest measured Accuracy: 59.49%

Highest measured Weighted F1: 59.25%

Works with the sparse TF-IDF representation.

Simple and computationally efficient for text classification.

Produces a reproducible trained model that has been saved as *.pkl.*

5. Evaluation Metrics

The trained models were evaluated on the unseen test set using Accuracy, Precision, Recall, and F1-score. Both macro-averaged and weighted-averaged metrics were considered to understand overall and class-level performance.

For the final Multinomial Naive Bayes model, the evaluation was performed on 1,817 test samples.

Final Model Evaluation

Metric	**Score**
Accuracy	59.49%
Macro Precision	59.34%
Macro F1-score	59.40%
Weighted Precision	59.16%
Weighted Recall	59.43%
Weighted F1-score	59.49%

Class-wise Performance

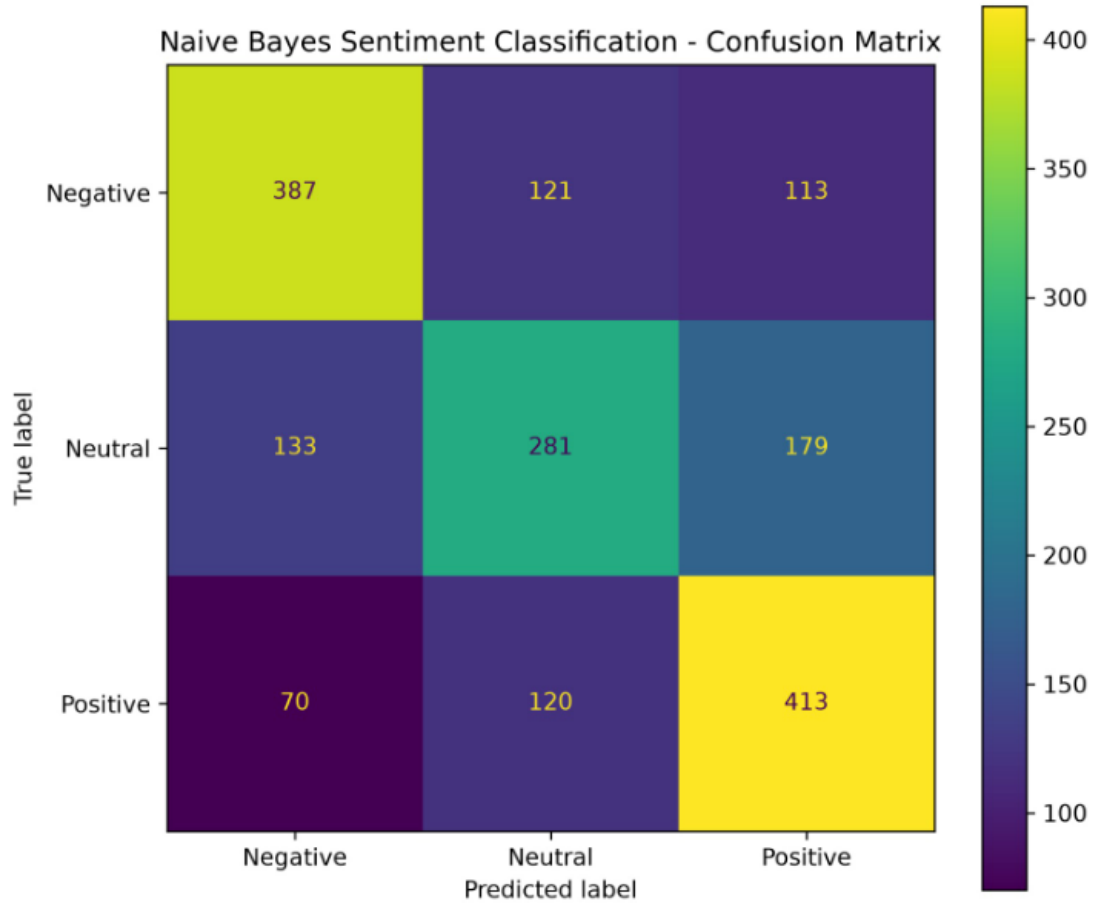
Sentiment	Precision	Recall	F1-score	Support
Negative	65.59%	62.32%	63.91%	621
Neutral	53.83%	47.39%	50.40%	593
Positive	58.58%	68.49%	63.15%	603

The results show that the model performs differently across sentiment classes. The Positive class has the highest recall, while the Neutral class has the lowest recall. This indicates that neutral posts are comparatively more difficult for the model to distinguish from positive and negative posts.

6. Confusion Matrix

A confusion Matrix was used to examine how the model's predictions where distributed across the three sentiment classes

Actual / predicted	Negative	Neutral	Positive
Negative	387	121	113
Neutral	133	281	179
Positive	70	120	413



Interpretation

- 387 Negative posts were correctly classified as Negative.
- 281 Neutral posts were correctly classified as Neutral.
- 413 Positive posts were correctly classified as Positive.
- The largest individual confusion was Neutral → Positive: 179 posts.
- Another significant confusion was Neutral → Negative: 133 posts.

7. Error Analysis

A total of 736 out of 1,817 test samples were misclassified by the final Multinomial Naive Bayes model. The misclassified samples were examined to identify common error patterns

Major Error Patterns

Actual	Predicated	Errors	% of Errors
Neutral	Positive	179	24.32%
Neutral	Negative	133	18.07%
Negative	Neutral	121	16.44%
Positive	Neutral	120	16.30%
Negative	Positive	113	15.35%
Positive	Negative	70	9.51%

The error analysis indicates that the major challenge is distinguishing Neutral content from Positive and Negative content. Several misclassified examples contained contextual, implicit, mixed, or ambiguous sentiment that cannot always be identified from individual words alone.

Examples of misclassified posts included sports updates, religious statements, criticism, frustration, announcements, and statements containing contextual sentiment. These cases demonstrate that purely TF-IDF-based classification can have difficulty capturing deeper semantic meaning, sarcasm, context, and mixed sentiment.

Conclusion

This project developed an end-to-end NLP-based sentiment classification pipeline for social media text. The dataset was first

audited and cleaned through a structured preprocessing process. To reduce data leakage caused by repeated text records, a group-aware train-test split was applied.

TF-IDF with unigrams and bigrams was then used to transform the cleaned text into numerical features. Three machine learning models—Logistic Regression, Linear SVM, and Multinomial Naive Bayes—were trained and evaluated using the same test set.

Among the evaluated configurations, Multinomial Naive Bayes achieved an accuracy of 59.49% and a weighted F1-score of 59.25%. The confusion matrix and error analysis showed that Neutral sentiment was comparatively difficult to classify, with frequent confusion between Neutral and the other sentiment classes.

The final trained model and TF-IDF vectorizer were saved as reusable model files, providing a reproducible foundation for further development of the semantic classification layer.